

크루온 CrewOn — 모임 운영 플랫폼

백엔드 4명 · 4주 · 웹 퍼스트 | 이 문서 하나로 "뭘 만들고, 내가 뭘 맡고, 당장 뭘 준비하는지"를 파악할 수 있게 정리했습니다.

1. 우리가 만드는 것 (30초 요약)

러닝크루·독서모임 같은 취미 모임은 지금 인스타(홍보) + 오픈카톡(소통) + 구글폼(신청) + 구글시트(출석·정산)로 수동 운영됩니다. 우리는 그중 운영 부분만 대체하는 크루장용 백오피스를 만듭니다. 모임을 홍보하는 곳(당근·밴드)과 경쟁하지 않고, 신청 링크부터 시작되는 그 다음 전 과정 — 선착순 신청, 대기열, 승급, 출석, 노쇼 관리 — 를 자동화합니다.

시나리오로 이해하기. 금요일 저녁, "한강 나이트러너스" 크루장 지수가 다음 주 세션 모집을 연다. 3분 만에 정원 20명 마감 — 예전엔 구글폼 21번째 응답자에게 사과 DM을 보냈지만, 지금은 초과 신청자가 자동으로 대기열에 선다. 화요일에 참가자 1명이 취소하자 대기 1번에게 승급 제안 알림이 가고, 제한시간 내 수락하지 않자 다음 순번으로 자동으로 넘어간다. 목요일 폭염주의보가 발령되자 대시보드에 "19:30 → 21:00 연기 제안" 카드가 뜨고, 지수는 클릭 한 번으로 전원에게 공지를 보낸다. 세션 당일, 지수 폰 화면의 QR을 현장에서 스캔한 사람만 출석 처리되고 안 나온 2명은 노쇼로 기록된다 — 노쇼 2회면 다음 세션 신청이 제한된다.

위 시나리오 한 단락에 우리 MVP 기능 전부와 기술 스택 전부(동시성·Redis·Kafka·WebSocket·외부 API)가 들어 있습니다.

2. 왜 이 주제인가

- 실용성 — 러닝크루·독서모임은 지금 유행 중인데 운영 도구가 없어 진짜로 수동 운영된다. 프로젝트 기간 중 실제 모임 1팀에 파일럿 배포가 목표 (러닝크루 우선, 안 되면 학원 스터디 모임)
- 기술 필연성 — 요구 스택 전부가 도메인에서 자연스럽게 나온다. "모집 오픈 순간 폭주"가 동시성 제어·K6의 명분, 신청 이후 후속 처리가 Kafka, 실시간 현황이 WebSocket
- 시의성 — 2026년 역대급 폭염. 기상청 특보 연동 기능이 심사 순간 와닿는다
- 포지셔닝 — 서비스 정체성은 범용 모임 플랫폼, 데모·발표는 요구사항이 가장 극적인 러닝크루로 고정 ("가로로 설계, 세로로 보여준다")

3. 서비스 흐름 (두 사용자의 여정)

크루원 (모바일 웹)

신청 링크 클릭 → 카카오키페 가입 → 세션 페이지에서 실시간 잔여 슬롯 확인 → 선착순 신청 → (자리 있으면) 즉시 확정 / (마감이면) 대기열 자동 등록 → 취소 발생 시 승급 제안 수신 → 제한시간 내 수락 → 확정 → 전날·3시간 전 리마인드 수신 → 당일 크루장 QR 스캔으로 출석

크루장 (데스크톱 웹)

모임 개설 → 세션 등록(일시·집결지·정원·노쇼 규칙) → 신청 링크를 인스타·오픈카톡에 공유 → 대시보드에서 신청·대기·취소·승급이 실시간 피드로 흐르는 것을 관찰 → 특보 발령 시 연기 제안 카드 확인·원클릭 일괄 공지 → 당일 QR 게시 → 종료 후 출석·노쇼 자동 집계 확인

핵심 용어 (팀 공용어)

용어	뜻
크루	모임 단위. category(러닝/독서/스터디/풋살)를 가짐 — 도메인 모델은 범용

용어	뜻
세션	크루가 여는 1회 모임. 일시·집결지·정원을 가짐 (장소 예약·중개는 스코프 아웃, 우리는 정보만 기록)
선착순 신청	원자적 판정으로 즉시 확정 또는 대기열 등록. "신청 실패 화면"은 구조적으로 없음 — 경쟁 패자는 대기열로
승급 제안	최소로 빈자리 발생 시 대기 1번에게 가는 제한시간부 제안(Redis TTL). 미수락 시 다음 순번 자동 이관
노쇼	확정 후 미출석. 2회 누적 시 신청 제한 — 우리 서비스의 핵심 차별점이라 정체성(필수 가입)이 필요
동적 QR	크루장 화면에서 30초마다 갱신되는 서명(HMAC) QR. 현장에서만 스캔 가능해 대리 출석 차단
특보 연동	기상청 API 폴링 → 새 특보 감지 시 Kafka 이벤트 → 야외 카테고리 세션에 연기 제안 + 일괄 알림

4. 아키텍처 한눈에 (MSA 4서비스, 1인 1서비스)

판정(신청 확정/대기)은 동기로 즉시 응답하고, 그 이후의 모든 일(알림·순번 재계산·피드·통계)은 Kafka 이벤트로 비동기 처리합니다. 실시간 화면 갱신은 WebSocket이 담당합니다.

서비스	담당	한 줄 정의
Auth	A	"누구인지"를 책임진다 — 카카오톡 로그인, JWT, 역할, 노쇼 이력 프로필
모임·세션	B	"무엇이 열리는지"를 책임진다 — 크루/세션 관리, 대시보드 조회, 기상 특보 감지
신청·참가	C	"누가 참가하는지"를 책임진다 — 선착순 판정, 대기열, 승급, QR 출석 (동시성의 심장)
알림·실시간	D	"어떻게 알리는지"를 책임진다 — Kafka 컨슈머, 알림 발송, WebSocket 허브, AI 제안(스트레치)

- 주요 이벤트: session.applied → participant.confirmed → waitlist.promoted → participant.noshow / weather.alert
- 서비스 간 동기 호출은 RestTemplate (예: 신청 서비스가 세션 서비스에 정원 조회)
- 인프라: 로컬 Docker Compose(MySQL·Redis·Kafka)로 개발 → GitHub Actions CI/CD → AWS EKS 배포

5. 역할 분담 — 내가 맡은 것과 미리 공부할 것

담당	만드는 것 (MVP 기준)	미리 공부/준비할 것
A — Auth 서비스	카카오 OAuth2 로그인·간편가입, JWT 발급/갱신, 역할(크루장·운영진·크루원), 노쇼 이력 프로필 API. 모든 서비스가 이 토큰 검증에 의존하므로 W1에 가장 먼저 관통되어야 함	Spring Security 필터체인, OAuth2 클라이언트 등록(카카오 개발자 앱 생성 — W1 첫날), JWT 구조와 갱신 전략
B — 모임·세션 서비스	크루/세션 CRUD, 카테고리 확장 필드, 크루장 대시보드 조회(QueryDSL 동적 필터), 기상청 특보 폴러(스케줄러 + 분산락 + RestTemplate)	QueryDSL 동적 쿼리, 공공데이터포털 가입 + 기상청 특보/단기예보 API 활용신청(승인 대기 있으니 W1 첫날), 스케줄러 중복 실행 문제
C — 신청·참가 서비스	선착순 원자 판정(동시성 5종 실험 주도), 대기열, 승급 상태머신(Redis TTL), 동적 QR 발급/검증, 출석·노쇼 판정. 기술 난이도 최상 — 발표 하이라이트의 주인공	Redis 분산락(Redisson), DB 비관적/낙관적 락, 트랜잭션 경계, TTL 만료 처리 패턴, K6 스크립트 작성
D — 알림·실시간 서비스	Kafka 컨슈머 설계, 알림 발송(웹 푸시 + 이메일 — 카카오톡 알림톡은 사업자 인증 필요라 불가), WebSocket 허브(잔여 슬롯·순번·피드), AI 제안 컨슈머(W4 스트레치)	Kafka 기본기(토픽·파티션·컨슈머 그룹·재소비), WebSocket STOMP, Redis Pub/Sub로 다중 인스턴스 브로드캐스트

- 전원 공통: Docker Compose 로컬 환경, GitHub Actions, K6 시나리오 리뷰(주도는 C), 코드 리뷰 크로스 체크

- 인프라(EKS·CI/CD) 검임 담당자는 미정 — 첫 팀 회의에서 지정 필요 (7장 참고)

6. 일정 — 주차별 목표와 완료 조건

주차	전원이 이번 주에 끝내야 하는 것	완료 조건 (이게 되면 통과)
W1	ERD·API 스펙·이벤트 스키마 확정, Compose 환경, 4서비스 스켈레톤 + CI, 카카오 로그인 관통, 공공데이터 API 키 확보	가입 → JWT 발급 → 서비스 간 인증 호출 E2E 1개 통과
W2	모임/세션 CRUD, 신청 판정 v1(DB 락), 대기열 등록, Kafka 파이프라인 1개(신청→알림), WebSocket 잔여 슬롯	브라우저 2대에서 동시 신청 → 실시간 갱신 시연
W3	승급 상태머신(TTL), 동적 QR 출석, 노쇼 제한, 특보 연동, 대시보드, K6 1차 + 동시성 5종 실험, 고도화 Phase 1 착수	실험 데이터 표 완성, 대기 → 승급 → 수락 E2E 통과
W4	EKS 배포 + CD, K6 2차(개선 전후), 파일럿 1회, AI 세션 어시스턴트(스트레치), 발표 준비	운영 URL에서 파일럿 세션 1건 완주 (신청 → 출석)

7. 첫 팀 회의에서 정할 것 (미결정 목록)

안건	논의 포인트	현재 추천안
프론트엔드 수단	React 경험자가 팀에 있는가? 페이지는 6~7개(신청/결과/내 신청/QR 스캔/대시보드/세션 등록)로 최소화 전제	React+Vite 최소 SPA (경험자 전무 시 Thymeleaf 타협)
서비스명	“크루온(CrewOn)”은 가칭	팀 브레인스토밍
DB + ERD	MySQL 가정 중. W1 첫 이틀에 ERD 상세 설계 세션 필요	MySQL 8 + 서비스별 스키마 분리
크루원 가입 정책	노쇼 이력 관리(핵심 차별점)에 정제성이 필수 — 비회원 신청 허용 시 이름 바꿔 재신청으로 무력화됨	카카오 OAuth2 필수 가입
인프라 검임	EKS·CI/CD를 누가 검임할지. 본인 서비스 부담이 상대적으로 작은 담당 권장	A 또는 D 중 지원자
파일럿 섭외	아는 러닝크루가 있는 사람? 없으면 학원 내 스터디 모임으로 대체	W2까지 대상 확정

8. 발표에서 보여줄 것 (하이라이트 계획)

- 동시성 실험 리포트 — 같은 신청 API를 ① 제어 없음 ② synchronized ③ DB 비관적 락 ④ DB 낙관적 락 ⑤ Redis 분산락으로 교체하며 K6 “정원 20, 동시 500 요청” 실행. 오버셀 건수·p95·처리량 비교표로 최종 선택 근거 제시 (특히 ②가 파드 2개에서 무너지는 장면이 킬링 포인트)
- 라이브 데모 — 폰 2대로 승급 제한이 수락 안 되면 다음 순번으로 넘어가는 것을 실시간 시연
- 폭염 시나리오 — 특보 수신 → 대시보드 배너 → (스트레치) AI 대안 제안 카드 → 일괄 공지
- 파일럿 실사용 데이터 — “실제로 쓰였다”는 증거 (마감 시간, 출석률)

9. 백엔드 고도화 로드맵 (요약)

MVP 이후의 심화 계획. 상세 명세는 기획안 v3 문서 9장 참고.

단계	항목	핵심 이유
Phase 1 (W3~4, 프로젝트 내)	Transactional Outbox · 컨슈머 먹등성 · DLQ · 서킷브레이커(Resilience4j) · 캐시 스탬피드 방지 · HikariCP 튜닝	이벤트 유실/중복, 외부 API 장애로부터 핵심 기능 보호

단계	항목	핵심 이유
Phase 2 (발표 전 가점)	Prometheus+Grafana · K8s HPA · 무중단 배포 · 분산 트레이싱 · Testcontainers	부하 실험을 라이브 지표로 발표, "EKS 왜 썼나"의 실증
Phase 3 (수료 후)	API Gateway · rate limiting · JWT 로테이션 · 이벤트 스키마 버저닝 · 중앙 로깅	개인 포트폴리오 심화 과제

10. 리스크 요약

리스크	대응
카카오 알림톡 = 사업자 인증 필요	웹 푸시 + 이메일 대체 ("나에게 보내기"는 데모용)
Kafka·EKS 러닝커브	W1 로컬 walking skeleton 선관통, EKS는 W3 말 헬로월드 선배포
EKS 비용	소형 노드 2개, 시연 때만 스케일, 평시 로컬 Compose
공공 API 장애	타임아웃·재시도·서킷브레이커, 실패 시 기능 저하 운영
QR 부정(캡처 공유)	30초 로테이션 + 세션·시간 서명 검증
LLM 비용·지연 (스트레치)	3초 타임아웃 → 규칙 기반 폴백, 동일 토큰 캐싱, 일일 상한